EchoSign: An Integrated AI Platform for Real-Time American Sign Language Translation, Gesture Evaluation and Speech Synthesis

Report submitted in partial fulfillment of the requirement for the degree of

B. Tech.

In

Computer Science and Engineering

Under the Supervision of

Ms. Rita Nagar

Assistant Professor, C.S.E.

Submitted By:

Nidhi Singh (2303491530068)

Yuvraj Singh (2303491530128)

Rohit Kumar (2303491530090)

Prashant Kumar Singh (2303491530075)

Komal Goswami (2303491530061)

Department of Computer Science & Engineering

Maharana Pratap Engineering College, Kanpur

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Session: 2026-27

DECLARATION

This is to certify that the Project Report entitled “EchoSign: An AI-Powered American Sign Language Translation System Using Deep Convolutional Neural Networks and Real-Time Speech Synthesis”, which is submitted in partial fulfilment of the requirement for the award of the degree of B.Tech. in Computer Science and Engineering to MPEC Kanpur, Dr. A.P.J. Abdul Kalam Technical University, Lucknow, comprises only original work and studies carried out by the students themselves. The matter embodied in this report has not been submitted for the award of any other degree.

We further declare that all source code, database architectures, algorithmic modules, and technical documentation presented in this report have been implemented in accordance with academic integrity guidelines. All external open-source libraries and scholarly references utilized have been duly credited and cited.

Date: ……

Place: Kanpur

Nidhi Singh (2303491530068)
Yuvraj Singh (2303491530128)
Rohit Kumar (2303491530090)
Prashant Kumar Singh (2303491530075)
Komal Goswami (2303491530061)

Approved By:

Ms. Rita Nagar
Project Guide
CSE Department
MPEC, Kanpur
Signature: …….

Mr. Milind Bhatt
HOD
CSE Department
MPEC, Kanpur
Signature: …….

CERTIFICATE OF APPROVAL

This is to certify that the Final Year Project Report entitled “EchoSign: An AI-Powered American Sign Language Translation System Using Deep Convolutional Neural Networks and Real-Time Speech Synthesis”, submitted by:

- **Nidhi Singh** (University Roll No: 2303491530068)
- **Yuvraj Singh** (University Roll No: 2303491530128)
- **Rohit Kumar** (University Roll No: 2303491530090)
- **Prashant Kumar Singh** (University Roll No: 2303491530075)
- **Komal Goswami** (University Roll No: 2303491530061)

in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering of Dr. A.P.J. Abdul Kalam Technical University, Lucknow, is a bonafide record of original work carried out by them under our guidance and supervision.

The project satisfies all academic ordinances and is hereby approved for final evaluation and university viva-voce.

Ms. Rita Nagar
Project Guide
CSE Department
MPEC, Kanpur
Signature: …….

Mr. Milind Bhatt
HOD
CSE Department
MPEC, Kanpur
Signature: …….

_____
External Examiner (Signature & Date)

ACKNOWLEDGEMENTS

We express our sincere gratitude to our respected project guide, Ms. Rita Nagar, Assistant Professor, Department of Computer Science & Engineering, Maharana Pratap Engineering College, Kanpur, for her continuous encouragement, expert guidance, and constructive suggestions throughout the development of EchoSign. Her insights into deep learning regularization and preprocessing parity were instrumental to our success.

We extend our gratitude to Mr. Milind Bhatt, Head of the Department of Computer Science & Engineering, for providing the necessary laboratory resources, high-speed computational facilities, and a collaborative environment.

We also convey our thanks to all faculty members and staff of the CSE Department for their support, and to our parents and peers for their patience and motivation throughout our academic journey.

Project Team Members:

- Nidhi Singh (2303491530068)
- Yuvraj Singh (2303491530128)
- Rohit Kumar (2303491530090)
- Prashant Kumar Singh (2303491530075)
- Komal Goswami (2303491530061)

TABLE OF CONTENTS

Title Page ..... **i**
Candidate Declaration ..... **ii**
Certificate of Approval ..... **iii**
Acknowledgements ..... **iv**
Table of Contents ..... **v**
List of Acronyms and Abbreviations ..... **vii**
List of Figures ..... **viii**
List of Tables ..... **ix**
Abstract ..... **x**
CHAPTER 1 — INTRODUCTION **1**
1.1 Problem Context and Assistive Computing 1
1.2 Linguistic Structure of ASL Fingerspelling 2
1.3 Motivation and Real-World Impact 3
1.4 Project Objectives 4
1.5 Scope and Operational Boundaries 5
CHAPTER 2 — LITERATURE SURVEY AND RELATED WORK **6**
2.1 Evolution of Sign Language Recognition Systems 6
2.2 Review of Primary Recognition Paradigms 7
2.3 Critical Identification of Research Gaps 11
2.4 Comparative Summary of Existing Approaches vs. EchoSign 12
CHAPTER 3 — SYSTEM REQUIREMENTS AND SPECIFICATIONS **13**
3.1 Functional Requirements 13
3.2 Non-Functional Requirements 14
3.3 Hardware Requirements 15
3.4 Software Requirements and Development Tools 16
CHAPTER 4 — MATHEMATICAL FORMULATION AND METHODOLOGY **17**
4.1 Spatial Aspect Ratio and Center-Square Cropping Formulation 17
4.2 Mathematical Model of Bilinear Interpolation 18
4.3 Forward Convolutional Propagation and Tensor Arithmetic 19
4.4 Loss Function and Backpropagation Optimization 20
4.5 Softmax Probability Distribution and Statistical Thresholding 21
4.6 Float16 Quantization Mathematics 22
CHAPTER 5 — SYSTEM ARCHITECTURE AND DETAILED DESIGN **23**
5.1 Three-Tier Decoupled System Architecture 23
5.2 Data Flow Architecture (DFD Level 0 and Level 1) 24
5.3 Deep CNN Architecture Specification and Layer Parameters 26
5.4 Preprocessing and Normalization Pipeline 27
5.5 RESTful API Specifications and Interface Contracts 28
5.6 Client-Side Architecture, Canvas ROI, and Speech Synthesis 29
CHAPTER 6 — IMPLEMENTATION DETAILS **31**
6.1 Dataset Acquisition, Audit, and Partitioning 31
6.2 Preprocessing Engine Implementation 32

6.3 Model Training Pipeline and Callbacks 33
6.4 TFLite Quantization and Compression Pipeline 34
6.5 Flask Application Controller and Serverless Rewrites 35
6.6 Client Application and Web Speech Integration 36
CHAPTER 7 — RESULTS, TESTING AND PERFORMANCE EVALUATION 37
7.1 Experimental Training Convergence and Metrics 37
7.2 Evaluation on Held-Out Test Set (96.75% Accuracy) 38
7.3 Per-Class Performance and Confusion Matrix Analysis 39
7.4 Inference Latency and System Resource Accounting 41
7.5 Automated Software Testing Suite Matrix 42
CHAPTER 8 — PROJECT TIMELINE AND RESEARCH CONTRIBUTIONS 44
8.1 Time-Bound Activity Schedule (Weeks 1 to 13) 44
8.2 Allocation of Responsibilities Across 5 Team Members 45
8.3 Division into Two Distinct Research Paper Tracks 46
CHAPTER 9 — CONCLUSION AND FUTURE SCOPE 47
9.1 Summary of Achievements 47
9.2 Technical Limitations 48
9.3 Directions for Future Work 48
BIBLIOGRAPHY / REFERENCES 50

LIST OF ACRONYMS AND ABBREVIATIONS

Acronym	Full Form	Acronym	Full Form
ASL	American Sign Language	RGB	Red Green Blue Color Space
CNN	Convolutional Neural Network	ROI	Region of Interest
CPU	Central Processing Unit	SLA	Service Level Agreement
DFD	Data Flow Diagram	TTS	Text-to-Speech
FPS	Frames Per Second	UI / UX	User Interface / User Experience
GPU	Graphics Processing Unit	VCF	Vercel Cloud Functions
HOD	Head of the Department	VGG	Visual Geometry Group
JSON	JavaScript Object Notation	W3C	World Wide Web Consortium
MPEC	Maharana Pratap Engineering College	WebRTC	Web Real-Time Communication
REST	Representational State Transfer	WSGI	Web Server Gateway Interface

ABSTRACT

Individuals who are deaf or hard-of-hearing encounter pervasive communication barriers in social, commercial, healthcare, and educational environments due to low rates of sign language comprehension among the hearing majority. Certified human sign language interpreters are resource-constrained and costly, making continuous translation impractical for spontaneous everyday encounters. Existing automated systems frequently rely on specialized sensory hardware or heavyweight server-side deep learning pipelines that incur latency and cloud hosting costs.

This project presents EchoSign, an integrated, end-to-end web-based assistive communication platform engineered for real-time American Sign Language (ASL) fingerspelling recognition, interactive learning, and verbal speech synthesis. The system classifies hand gestures across 29 discrete categories—including all 26 standard alphabet postures (A through Z) and 3 control gestures (space, delete, and neutral resting state).

The machine learning core utilizes a deep Convolutional Neural Network trained on a curated corpus of 20,300 color images (700 images per class) derived from the benchmark ASL Alphabet dataset. To eliminate the domain gap between model training and browser deployment, the system enforces mathematical preprocessing parity: webcam video streams and training loaders apply an identical center-square aspect-ratio crop and Pillow-based bilinear interpolation down to a uniform 64×64 pixel grid.

Float16 quantization reduces model parameters to a 4.29 MB TensorFlow Lite artifact, enabling fast cold starts and real-time CPU inference on serverless infrastructure. Comprehensive evaluation over a held-out test set of 2,030 unseen images demonstrates 96.75% classification accuracy with a median execution latency of 16.10 ms on standard CPUs (24.70 ms p95). The client application pairs an in-browser Region-of-Interest (ROI) canvas guide with temporal debounce filtering and native Web Speech API audio synthesis.

Keywords: *American Sign Language, Convolutional Neural Networks, Deep Learning, TensorFlow Lite, Model Quantization, Bilinear Resampling, Web Speech API, Assistive Technology, Human-Computer Interaction.*

CHAPTER 1
INTRODUCTION

1.1 Problem Context and Assistive Computing

Language is the primary instrument through which human beings express intent, build social bonds, access healthcare, acquire education, and participate in civic governance. Over 70 million individuals worldwide utilize sign language as their primary language. In North America and various international communities, American Sign Language (ASL) serves as the primary linguistic medium for deaf and hard-of-hearing people. Despite its richness, fewer than 1% of the hearing population possesses working literacy in sign language. Consequently, deaf individuals routinely experience severe communication barriers when accessing essential services such as emergency triage, hospital consultations, educational lectures, public transit ticketing, and retail transactions. Certified human sign language interpreters are scarce and financially demanding, making full-time human translation unfeasible for casual, day-to-day encounters. Assistive computing offers an opportunity to bridge this divide through autonomous, software-driven translation. By leveraging ubiquitous consumer webcams, modern web browsers, and deep learning models, visual gestures can be converted into spoken English instantly without imposing wearable hardware on the user.

1.2 Linguistic Structure of ASL Fingerspelling

American Sign Language comprises two distinct lexical components: core conceptual signs and manual fingerspelling (the manual alphabet). Fingerspelling plays a fundamental role in sign language discourse: it is utilized to communicate proper nouns, personal names, geographic locations, technical terminology, and brand names that lack dedicated root signs. The ASL alphabet is executed using a single dominant hand held upright within a signing space located between the shoulder and chest. Of the 26 alphabet postures, 24 letters (A through I, K through Y) are completely static configurations defined by relative finger extensions, thumb opposition, and knuckle flexion. The letters J and Z involve motion trajectories. In automated recognition, static fingerspelling provides an ideal foundation for high-precision assistive computer vision: each letter is an invariant geometric shape that can be classified from individual 2D camera frames.

1.3 Motivation and Real-World Impact

The overarching motivation behind EchoSign is to create an assistive translation technology that prioritizes user autonomy, affordability, and zero friction. Wearable sensor gloves demand that deaf individuals carry, wear, and power specialized electronic hardware—an approach rejected by the deaf community as stigmatizing and impractical. Conversely, high-parameter deep learning architectures requiring desktop GPUs cannot run on portable consumer laptops or mobile browsers during field encounters. EchoSign combines a lightweight, quantized Convolutional Neural Network with a serverless web architecture and client-side speech synthesis, transforming any smartphone, tablet, or laptop into a sign language communicator without requiring software installations, specialized sensors, or cloud subscription fees.

1.4 Project Objectives

1. Build and train a custom Convolutional Neural Network tailored for 29-class ASL alphabet classification, incorporating double-convolution feature extraction, batch normalization, and dropout regularization.

2. Train the model on a balanced corpus of 20,300 color images (700 images per class) to achieve an overall test accuracy exceeding 95% across all 26 alphabet letters and 3 operational control gestures (del, nothing, space).
3. Implement an identical image preprocessing transformation across both training data loading and real-time inference using center-square cropping and Pillow-based bilinear interpolation, eliminating domain shift and aspect-ratio distortion.
4. Quantize model parameters to float16 precision and convert to TensorFlow Lite format, reducing model binary size below 5 MB to ensure fast cold starts and sub-25 ms execution times on standard serverless vCPUs.
5. Develop a responsive, accessible web portal using Flask and Bootstrap 5 supporting image upload, interactive live camera processing with Region-of-Interest (ROI) alignment guides, and video analysis.
6. Implement client-side auditory vocalization using the browser-native Web Speech API without external paid third-party voice APIs.
7. Integrate an interactive ASL Alphabet Reference Guide displaying standardized hand postures and anatomical descriptions to serve as an educational resource for beginners.

1.5 Scope and Operational Boundaries

The functional scope of EchoSign focuses on static ASL manual fingerspelling. Dynamic letters (J and Z) are evaluated via representative static pause postures accompanied by an automatic contextual advisory notice on the interface. The system transliterates fingerspelled signs into English letters and words; it does not attempt full semantic parsing of continuous multi-hand grammatical sign sentences. The web platform operates over modern web browsers supporting WebRTC camera permissions and HTML5 canvas manipulation, requiring no native executable downloads or proprietary external hardware.

CHAPTER 2
LITERATURE SURVEY AND RELATED WORK

2.1 Evolution of Sign Language Recognition Systems

Research into automated sign language recognition has progressed through three technological eras: the Sensor and Wearable Era (1990s–2005) characterized by hardware-intensive data gloves; the Classical Computer Vision Era (2005–2014) focused on camera-based capture combined with handcrafted feature engineering and skin-color modeling; and the Deep Learning and Edge AI Era (2015–Present) dominated by end-to-end convolutional neural networks, spatial-temporal graph networks, and lightweight quantized edge models.

2.2 Review of Primary Recognition Paradigms

1. **Wearable Sensor Gloves:** Measure digit curvature via flex sensors and wrist tilt via IMUs. Provide clean electrical signals immune to room lighting, but are expensive, physically fragile, cumbersome, and rejected by users for everyday public interactions.
2. **Classical Computer Vision (Skin Segmentation & Contours):** Isolate hands using YCbCr or HSV thresholding and classify with HOG/SVM. Highly sensitive to illumination shifts, skin tone variations, and background clutter, dropping below 70% accuracy in real-world environments.
3. **Deep Convolutional Neural Networks (CNNs):** Learn spatial features directly from raw pixels. Resilient to background variations, but standard architectures (VGG-16, ResNet-50) require desktop GPUs and exceed cloud serverless package limits.
4. **Coordinate Landmark Regressors (MediaPipe):** Extract 21 3D knuckle keypoints. Computationally lightweight, but experience severe tracking failures during self-occluding closed fists ('E', 'M', 'N', 'S', 'T'). Direct CNN feature extraction preserves the global textural and shadow cues necessary to distinguish subtle knuckle positions.

2.3 Critical Identification of Research Gaps

Our systematic review identified four critical technical gaps: (1) The Preprocessing Inconsistency Gap: disparity between OpenCV training loaders and Pillow production web APIs alters pixel gradients, degrading field accuracy; (2) The Aspect Ratio Distortion Gap: direct scaling of rectangular webcam feeds squashes the hand, warping finger geometry; (3) The Deployment Footprint Gap: deep models exceed 25 MB and require multi-hundred MB runtimes, failing serverless deployment constraints; (4) The Assistive Loop Gap: prototypes lack sentence buffers, backspace deletion, and speech synthesis.

CHAPTER 3
SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1 Functional Requirements

- **FR-01:** Accept static image uploads (JPEG, PNG, WebP) up to 10 MB.
- **FR-02:** Stream live webcam video via browser WebRTC at 400–600 ms sampling intervals.
- **FR-03:** Execute automated center-square cropping and bilinear resampling to 64x64 pixels.
- **FR-04:** Classify gestures across 29 classes with statistical confidence scores.
- **FR-05:** Render top-1 prediction, confidence score, posture description, and top-5 alternatives.
- **FR-06:** Maintain an editable text composition buffer with space and backspace handling.
- **FR-07:** Vocalize composed sentences via the native browser Web Speech API.

3.2 Non-Functional Requirements

- **Performance SLA:** Serverless inference latency must remain strictly below 50 milliseconds per frame.
- **Package Constraint:** Deployment artifact including model must not exceed 50 MB (Vercel cap is 250 MB).
- **Target Accuracy:** Overall test set accuracy exceeding 95% on at least 2,000 unseen evaluation samples.
- **Privacy by Design:** Ephemeral in-memory image processing with zero server-side storage of user frames.
- **Cross-Platform Compatibility:** Seamless responsive execution across Chromium, WebKit, and Gecko browser engines.

CHAPTER 4
MATHEMATICAL FORMULATION AND METHODOLOGY

4.1 Spatial Aspect Ratio and Center-Square Cropping Formulation

Given input frame I of width W and height H, non-uniform scaling to 64x64 produces unequal scaling factors sx = 64/W and sy = 64/H, distorting finger geometry. EchoSign establishes maximum square dimension D = min(W, H). Spatial offsets are computed as:

```
x_start = floor((W - D) / 2),    y_start = floor((H - D) / 2)
```

The square sub-image I_crop in R^(D x D x 3) is extracted across:

```
I_crop(i, j, c) = I(y_start + i,  x_start + j,  c),  for all i, j in {0, ..., D-1}.
```

4.2 Mathematical Model of Bilinear Interpolation

Continuous target coordinates (x, y) map to four surrounding discrete source pixels (x1, y1), (x1, y2), (x2, y1), (x2, y2). With fractional offsets alpha = x - x1 and beta = y - y1, the resampled intensity is:

```
f(x, y) = (1 - alpha)(1 - beta) I(y1, x1) + alpha(1 - beta) I(y1, x2) + (1 -
alpha)beta I(y2, x1) + alpha beta I(y2, x2)
X = f(x, y) / 255.0, where X in [0.0, 1.0]^(1 x 64 x 64 x 3).
```

4.3 Forward Convolutional Propagation and Softmax Classification

Within each convolutional layer l, a bank of 3x3 filters processes incoming feature maps:

```
Z_k^(l)(i, j) = sum( W_k^(l) * A^(l-1) ) + b_k^(l)
```

Non-linearity is introduced via ReLU: A_k^(l) = max(0, Z_k^(l)). Max-pooling applies 2x2 windows with stride 2. The output logits z in R^29 are mapped to probabilities via Softmax:

```
p_i = exp(z_i) / sum(exp(z_j)),   for all i in {1, ..., 29}
```

Confidence C = max(p_i). Acceptance threshold T_conf = 0.50.

4.4 Float16 Quantization Mathematics

Weights are converted from IEEE 754 32-bit single-precision to 16-bit half-precision (1 sign bit, 5 exponent bits, 10 mantissa bits). Dynamic range spans 6.10e-5 to 6.55e+4, preserving convolutional filter precision while reducing model storage by 84% (from 26.99 MB to 4.29 MB).

CHAPTER 5
SYSTEM ARCHITECTURE AND DETAILED DESIGN

5.1 Three-Tier Decoupled System Architecture

- **Tier 1:** Client Presentation Layer (HTML5, Bootstrap 5, WebRTC camera capture, 70% ROI canvas guide, Web Speech TTS).
- **Tier 2:** Application Service Layer (Flask WSGI handler, base64 decoding, Pillow aspect cropping and bilinear resampling, lexical dictionary mapping).
- **Tier 3:** Deep Inference Engine (TensorFlow Lite runtime via ai-edge-litert, 4.29 MB float16 double-conv model, 29-class probability vector generation).

5.2 Data Flow Architecture (DFD Level 0 and Level 1)

Level 0 Context Diagram depicts user webcam input flowing into EchoSign and returning English text, confidence, and speech. Level 1 Functional DFD defines 5 coordinated processes: (1.0) Frame Ingestion & Validation; (2.0) Preprocessing & Resampling; (3.0) TFLite Inference Execution; (4.0) Lexical Lookup & Confidence Filtering; (5.0) Client Sentence Assembly & Speech Vocalization.

CHAPTER 6
IMPLEMENTATION DETAILS

6.1 Dataset Curation and Partitioning

Curated 20,300 color images (700 images per class across 29 categories) from the ASL Alphabet dataset. Partitioned into 15,529 training images (76.5%), 2,741 validation images (13.5%), and 2,030 held-out test images (10.0%).

6.2 Preprocessing Engine Implementation

Centralized in `src/preprocessing.py`. Implements aspect ratio detection, min(W, H) center cropping, Pillow BILINEAR resizing to 64x64, float32 normalization to [0.0, 1.0], and batch expansion to (1, 64, 64, 3).

6.3 Deep CNN Training Procedure

Trained over 14 epochs using Adam optimizer (initial learning rate 0.001, batch size 64). Utilized ModelCheckpoint on validation accuracy, ReduceLROnPlateau (factor 0.5, patience 2), and EarlyStopping (patience 4). Final validation accuracy reached 96.70%.

6.4 TFLite Quantization and Serverless Cloud Integration

Quantized using TFLiteConverter with float16 supported types. Model file size compressed to 4.29 MB. Deployed on Vercel using `api/index.py` with custom `VercelPathMiddleware` route rewrites.

CHAPTER 7
RESULTS, TESTING AND PERFORMANCE EVALUATION

7.1 Training Convergence and Test Accuracy

Training loss decreased smoothly from 3.36 to 0.08, while validation accuracy reached 96.70%. On the held-out test set of 2,030 unseen images, the quantized model achieved 96.75% accuracy (1,964 correct classifications out of 2,030).

7.2 Per-Class Confusion Analysis

28 out of 29 classes achieved recognition accuracies between 92.9% and 100.0%. All difficult closed-fist categories ('A', 'E', 'M', 'N', 'S', 'T') demonstrated high discrimination (94.3%–100.0%). Minor confusion occurred between 'R' (80.0%) and 'U' (75.7%) due to crossed vs. parallel finger overlap at 64x64 resolution.

7.3 Latency Accounting and Automated Testing

Median serverless execution latency clocked at 24.70 ms on Vercel (16.10 ms on local CPU), well within the 50 ms SLA. Automated test suite (8 test cases covering preprocessing, shape invariance, API routes, and model inference) passed with 100% success rate.

CHAPTER 8
PROJECT TIMELINE AND RESEARCH CONTRIBUTIONS

8.1 13-Week Time-Bound Engineering Schedule

Project executed across 13 weeks: Weeks 1–2 (Requirements & Literature); Weeks 3–5 (Dataset & Preprocessing Parity); Weeks 6–8 (CNN Training & Quantization); Weeks 9–11 (Flask API, Web Speech & Cloud Deployment); Weeks 12–13 (Validation, Documentation & Research Papers).

8.2 Division into Two Research Paper Tracks

In accordance with MPEC CSE departmental guidelines:

- **Research Track 1 (Nidhi Singh & Yuvraj Singh):** Deep CNN Optimization, Preprocessing Parity, and Float16 Quantization for High-Accuracy ASL Alphabet Recognition.
- **Research Track 2 (Rohit Kumar, Prashant Kumar Singh, Komal Goswami):** Serverless Edge Architecture for Real-Time Sign Language Translation with In-Browser ROI Extraction and Native Speech Synthesis.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Summary of Contributions

EchoSign provides an accessible, high-accuracy (96.75%), zero-installation sign language translation web service. Eliminating the preprocessing gap through center-square cropping and bilinear resampling ensures robust real-world webcam performance. Float16 quantization enables sub-25 ms serverless execution under 5 MB storage footprints.

9.2 Limitations and Future Enhancements

Current limitations include static evaluation of dynamic letters (J/Z) and single-hand fingerspelling scope. Future work will explore hybrid MediaPipe landmark fusion, lightweight temporal GRU video models for dynamic sign trajectories, and multi-dialect expansion to Indian Sign Language (ISL).

BIBLIOGRAPHY / REFERENCES

[1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.

[2] Akash Nagaraj. (2018). ASL Alphabet Dataset. Kaggle Datasets, DOI: 10.34740/KAGGLE/DSV/26430.

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press, Cambridge, MA.

[4] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. NeurIPS, 25, 1097–1105.

[5] Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861.

[6] Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.

[7] David, R., et al. (2021). TensorFlow Lite: On-device machine learning framework. IEEE Micro, 41(4), 106–114.

[8] Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition: a survey. AI Review, 43(1), 1–54.

[9] Bantupalli, K., & Xie, Y. (2018). ASL recognition using deep learning. IEEE Big Data, pp. 4396–4399.

[10] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.

[11] Srivastava, N., et al. (2014). Dropout: a simple way to prevent overfitting. JMLR, 15(1), 1929–1958.

[12] W3C Speech API Community Group. (2020). Web Speech API Specification. World Wide Web Consortium.

[13] Sturman, D. J., & Zeltzer, D. (1994). A survey of glove-based input. IEEE Computer Graphics and Applications, 14(1), 30–39.

[14] Ahmed, M. A., et al. (2018). Smart glove for sign language translation using Arduino and flex sensors. IJET, 7(4.28), 123–128.

[15] Halder, A., & Tayade, A. (2021). Real-time sign language fingerspelling recognition using MediaPipe. IJACSA, 12(8), 532–539.

[16] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

[17] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

[18] IEEE Standard Association. (2017). IEEE Standard for Software and System Test Documentation. IEEE Std 829-2017.